

```

within Buildings.Fluid.Chillers;
model ElectricReformulatedEIR
  "Electric chiller based on the DOE-2.1 model, but with performance as a function of
  condenser leaving instead of entering temperature"
  extends Buildings.Fluid.Chillers.BaseClasses.PartialElectric(
    final QEva_flow_nominal = per.QEva_flow_nominal,
    final COP_nominal= per.COP_nominal,
    final PLRMax= per.PLRMax,
    final PLRMinUnl= per.PLRMinUnl,
    final PLRMin= per.PLRMin,
    final etaMotor= per.etaMotor,
    final mEva_flow_nominal= per.mEva_flow_nominal,
    final mCon_flow_nominal= per.mCon_flow_nominal,
    final TEvalvg_nominal= per.TEvalvg_nominal);

  parameter Buildings.Fluid.Chillers.Data.ElectricReformulatedEIR.Generic per
    "Performance data"
    annotation (choicesAllMatching = true,
      Placement(transformation(extent={{40,80},{60,100}})));

protected
  final parameter Modelica.Units.NonSI.Temperature_degC TConLvg_nominal_degC=
    Modelica.Units.Conversions.to_degC(per.TConLvg_nominal)
    "Temperature of fluid leaving condenser at nominal condition";

  Modelica.Units.NonSI.Temperature_degC TConLvg_degC
    "Temperature of fluid leaving condenser";
initial equation
  // Verify correctness of performance curves, and write warning if error is bigger than
  10%
  Buildings.Fluid.Chillers.BaseClasses.warnIfPerformanceOutOfBounds(
    Buildings.Utilities.Math.Functions.biquadratic(a=per.capFunT,
      x1=TEvalvg_nominal_degC, x2=TConLvg_nominal_degC),
    "Capacity as a function of temperature ",
    "per.capFunT");
equation
  TConLvg_degC=Modelica.Units.Conversions.to_degC(TConLvg);

  if on then
    // Compute the chiller capacity fraction, using a biquadratic curve.
    // Since the regression for capacity can have negative values (for unreasonable
    temperatures),
    // we constrain its return value to be non-negative. This prevents the solver to pick
    the
    // unrealistic solution.
    capFunT = Buildings.Utilities.Math.Functions.smoothMax(
      x1 = 1E-6,
      x2 = Buildings.Utilities.Math.Functions.biquadratic(a=per.capFunT, x1=TEvalvg_degC,
x2=TConLvg_degC),
      deltaX = 1E-7);
  /*    assert(capFunT > 0.1, "Error: Received capFunT = " + String(capFunT) + ".\n"
    + "Coefficient for polynomial seem to be not valid for the encountered
    temperature range.\n"
    + "Temperatures are TConLvg_degC = " + String(TConLvg_degC) + " degC\n"
    + "TEvalvg_degC = " + String(TEvalvg_degC) + " degC");
  */
  // Chiller energy input ratio biquadratic curve.
  EIRFunT = Buildings.Utilities.Math.Functions.biquadratic(a=per.EIRFunT,
x1=TEvalvg_degC, x2=TConLvg_degC);
  // Chiller energy input ratio bicubic curve
  EIRFunPLR = Buildings.Utilities.Math.Functions.bicubic(a=per.EIRFunPLR,
x1=TConLvg_degC, x2=PLR2);
  else
    capFunT = 0;
    EIRFunT = 0;
    EIRFunPLR = 0;
  end if;
  annotation (Icon(graphics={
    Rectangle(

```

```

        extent={{-104,66},{98,54}},
        lineColor={0,0,255},
        pattern=LinePattern.None,
        fillColor={0,0,255},
        fillPattern=FillPattern.Solid),
Rectangle(
    extent={{-2,54},{98,66}},
    lineColor={0,0,255},
    pattern=LinePattern.None,
    fillColor={255,0,0},
    fillPattern=FillPattern.Solid),
Rectangle(
    extent={{-44,52},{-40,12}},
    lineColor={0,0,0},
    fillColor={255,255,255},
    fillPattern=FillPattern.Solid),
Rectangle(
    extent={{-56,70},{58,52}},
    lineColor={0,0,0},
    fillColor={255,255,255},
    fillPattern=FillPattern.Solid),
Polygon(
    points={{-42,2},{-52,12},{-32,12},{-42,2}},
    lineColor={0,0,0},
    smooth=Smooth.None,
    fillColor={255,255,255},
    fillPattern=FillPattern.Solid),
Polygon(
    points={{-42,2},{-52,-10},{-32,-10},{-42,2}},
    lineColor={0,0,0},
    smooth=Smooth.None,
    fillColor={255,255,255},
    fillPattern=FillPattern.Solid),
Rectangle(
    extent={{-44,-10},{-40,-50}},
    lineColor={0,0,0},
    fillColor={255,255,255},
    fillPattern=FillPattern.Solid),
Rectangle(
    extent={{38,52},{42,-50}},
    lineColor={0,0,0},
    fillColor={255,255,255},
    fillPattern=FillPattern.Solid),
Ellipse(
    extent={{18,24},{62,-18}},
    lineColor={0,0,0},
    fillColor={255,255,255},
    fillPattern=FillPattern.Solid),
Polygon(
    points={{40,24},{22,-8},{58,-8},{40,24}},
    lineColor={0,0,0},
    smooth=Smooth.None,
    fillColor={255,255,255},
    fillPattern=FillPattern.Solid)),
Documentation(info="<html>
<p>
Model of an electric chiller, based on the model by
Hydeman et al. (2002) that has been developed in the CoolTools project
and that is implemented in EnergyPlus as the model
<code>Chiller:Electric:ReformulatedEIR</code>.
This empirical model is similar to
<a href=\"modelica://Buildings.Fluid.Chillers.ElectricEIR\">
Buildings.Fluid.Chillers.ElectricEIR</a>.
The difference is that to compute the performance, this model
uses the condenser leaving temperature instead of the entering temperature,
and it uses a bicubic polynomial to compute the part load performance.
</p>
<p>

```

This model uses three functions to predict capacity and power consumption:</p>

Function		
Description		
Formulation		
<code>capFunT</code>	Adjusts cooling capacity for current fluid temperatures	Biquadratic on <code>TConEnt</code> and <code>TEvaLvg</code>
<code>EIRFunPLR</code>	Adjusts EIR for the current PLR	Quadratic on PLR
<code>EIRFunT</code>	Adjusts EIR for current fluid temperatures	Biquadratic on <code>TConEnt</code> and <code>TEvaLvg</code>

These curves are stored in the data record `per` and are available from [Buildings.Fluid.Chillers.Data.ElectricReformulatedEIR](#). How they are used to compute the adjusted capacity and compressor power can be found in the documentation of [Buildings.Fluid.Chillers.BaseClasses.PartialElectric](#). Additional performance curves can be developed using two available techniques (Hydeman and Gillespie, 2002). The first technique is called the Least-squares Linear Regression method and is used when sufficient performance data exist to employ standard least-square linear regression techniques. The second technique is called Reference Curve Method and is used when insufficient performance data exist to apply linear regression techniques. A detailed description of both techniques can be found in Hydeman and Gillespie (2002).

The model takes as an input the set point for the leaving chilled water temperature, which is met if the chiller has sufficient capacity. Thus, the model has a built-in, ideal temperature control. The model has three tests on the part load ratio and the cycling ratio:</p>

```
PLR1 =min(QEva_flow_set/QEva_flow_ava, per.PLRMax);
```

ensures that the chiller capacity does not exceed the chiller capacity specified

by the parameter `per.PLRMax`.

The test

```
CR = min(PLR1/per.PLRMin, 1.0);
```

computes a cycling ratio. This ratio expresses the fraction of time that a chiller would run if it were to cycle because its load is smaller than the minimal load at which it can operate.

Note that this model continuously operates even if the part load ratio is below the minimum part load ratio.

Its leaving evaporator and condenser temperature can therefore be considered as an average temperature between the modes where the compressor is off and on.

The test

```
PLR2 = max(per.PLRMinUnl, PLR1);
```

computes the part load ratio of the compressor.

The assumption is that for a part load ratio below `per.PLRMinUnl`, the chiller uses hot gas bypass to reduce the capacity, while the compressor power draw does not change.

The electric power only contains the power for the compressor, but not any power for pumps or fans.

The model can be parametrized to compute a transient or steady-state response.

The transient response of the chiller is computed using a first order differential equation for the evaporator and condenser fluid volumes.

The chiller outlet temperatures are equal to the temperatures of these lumped volumes.

Optionally, the model can be configured to represent heat recovery chillers with a switchover option by setting the parameter `have_switchover` to `true`.

In that case an additional Boolean input connector `coo` is used.

The chiller is tracking a chilled water supply temperature setpoint at the

outlet of the evaporator barrel if `coo` is `true`.

Otherwise, if `coo` is `false`, the chiller is tracking a hot water supply temperature setpoint at the outlet of the condenser barrel.

See

[Buildings.Fluid.Chillers.Examples.ElectricEIR_HeatRecovery](https://modelica.org/doc/Bldg/Buildings.Fluid.Chillers.Examples.ElectricEIR_HeatRecovery)

for an example with a chiller operating in heating mode.

References

Hydeman, M., N. Webb, P. Sreedharan, and S. Blanc. 2002. Development and Testing of a Reformulated Regression-Based Electric Chiller Model. *ASHRAE Transactions*, HI-02-18-2.

Hydeman, M. and K.L. Gillespie. 2002. Tools and Techniques to Calibrate Electric Chiller Component Models. *ASHRAE Transactions*, AC-02-9-1.

revisions="<html>

March 12, 2015, by Michael Wetter:

Refactored model to make it once continuously differentiable.

This is for issue <https://github.com/lbl-srg/modelica-buildings/issues/>

```
373\">373</a>.  
</li>  
<li>  
Jan. 9, 2011, by Michael Wetter:<br/>  
Added input signal to switch chiller off.  
</li>  
<li>  
September 17, 2010, by Michael Wetter:<br/>  
First implementation.  
</li>  
</ul>  
</html>"));  
end ElectricReformulatedEIR;
```